AI, LLMs and RAG: Study Notes

A sample document for practicing LangChain document loading, chunking, embeddings and retrieval.

Chapter 1: Introduction to Artificial Intelligence

Artificial Intelligence (AI) is the field of computer science that focuses on building systems capable of performing tasks that normally require human intelligence. These tasks include understanding language, recognizing images, making decisions, and learning from experience.

AI is commonly divided into narrow AI and general AI. Narrow AI is designed for one specific task, such as spam filtering or recommending movies. General AI, which does not yet exist, would be able to perform any intellectual task a human can do.

Machine learning is a subset of AI where systems learn patterns from data instead of being explicitly programmed. Deep learning is a further subset that uses neural networks with many layers to learn complex representations.

Today, AI is used in healthcare for diagnosing diseases, in finance for detecting fraud, in education for personalized learning, and in everyday apps such as voice assistants and translation tools.

Chapter 2: Large Language Models

A Large Language Model (LLM) is a neural network trained on massive amounts of text so that it can understand and generate human language. Examples include GPT, Claude, Gemini, and Llama.

LLMs are built using the Transformer architecture, introduced in 2017. The key idea of the Transformer is the attention mechanism, which lets the model decide which words in a sentence are most relevant to each other.

LLMs have a limitation called the context window, which is the maximum amount of text the model can read at one time. If a document is longer than the context window, it must be split into smaller pieces.

Another limitation is that LLMs only know what they saw during training. They do not know about your private documents or about events that happened after their training cutoff. This is why techniques like Retrieval-Augmented Generation are so useful.

Chapter 3: Embeddings and Vector Databases

An embedding is a list of numbers that represents the meaning of a piece of text. Texts with similar meanings have embeddings that are close to each other in vector space. For example, the sentences 'I love dogs' and 'I adore puppies' would have very similar embeddings.

Embedding models, such as OpenAI's text-embedding models or open-source Sentence Transformers, convert text into these numerical vectors. The same model must be used for both documents and queries.

A vector database stores embeddings and allows fast similarity search. Popular options include FAISS, Chroma, Pinecone, and Weaviate. FAISS, developed by Meta, runs locally and is a great choice for learning and small projects.

Similarity between vectors is usually measured using cosine similarity or Euclidean distance. When a user asks a question, the question is converted into an embedding and the database returns the chunks with the closest vectors.

Chapter 4: Text Chunking

Chunking is the process of splitting a long document into smaller pieces called chunks. It is necessary because embedding models and LLMs have input size limits, and because smaller focused chunks give more precise search results.

The two most important chunking parameters are chunk_size and chunk_overlap. Chunk size is the maximum length of each chunk, usually measured in characters or tokens. Chunk overlap is the number of characters shared between consecutive chunks, which helps preserve context across boundaries.

In LangChain, the RecursiveCharacterTextSplitter is the most commonly recommended splitter. It tries to split on paragraphs first, then sentences, then words, so that chunks stay as meaningful as possible.

A common starting point is a chunk size of 500 to 1000 characters with an overlap of 50 to 200 characters. If chunks are too small, they lose context. If they are too large, search results become vague and expensive to process.

Other chunking strategies include splitting by fixed token count, splitting by Markdown headers, and semantic chunking, where the split points are chosen based on changes in meaning between sentences.

Chapter 5: Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, or RAG, is a technique that combines a search system with an LLM. Instead of relying only on the model's memory, RAG retrieves relevant information from your own documents and gives it to the model as context.

A typical RAG pipeline has these steps: first, load the documents; second, split them into chunks; third, convert the chunks into embeddings; fourth, store the embeddings in a vector database; fifth, at question time, retrieve the most relevant chunks; and sixth, pass the chunks and the question to an LLM to generate the final answer.

RAG reduces hallucinations because the model can base its answer on real source text. It also makes it easy to update knowledge: you simply add new documents to the database without retraining the model.

Good RAG systems also show their sources, so users can verify where an answer came from. In LangChain, the retrieved Document objects carry metadata such as the file name and page number for this purpose.

Chapter 6: Introduction to LangChain

LangChain is an open-source framework that makes it easier to build applications powered by language models. It provides ready-made components for loading documents, splitting text, creating embeddings, storing vectors, and building chains of LLM calls.

Key LangChain concepts include Document Loaders, Text Splitters, Embeddings, Vector Stores, Retrievers, and Chains. A Document Loader such as PyPDFLoader reads files into Document objects, each holding page_content and metadata.

A Retriever is an interface that takes a query and returns relevant documents. Any vector store can be turned into a retriever using the as_retriever method, and you can control how many results are returned with the k parameter.

LangChain is modular, so you can swap components easily. For example, you can start with FAISS and later move to Pinecone, or switch from one LLM provider to another, with only small changes in your code.